K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

KERNEL MONITOR

Linux System Resource Monitoring and Alert System

PROJECT REPORT
Operating Systems and Systems Programming | 25CS2104E
Academic Year: 2026–2027 | Term I

Team: TechNova

Team Member	Roll Number
M. Subhash	2520030391
D. Namith	2520030249
E. Yeshwanth	2520030539

Abstract

Kernel Monitor is a lightweight, terminal-based Ubuntu/Linux application developed in C to display current CPU utilization, RAM usage, root filesystem disk usage, total visible processes and one-minute system load average. It reads Linux kernel-provided interfaces including /proc/stat, /proc/meminfo and /proc/loadavg, counts numeric process directories under /proc, and uses the statvfs API for filesystem usage. The application refreshes every second and prints warnings if CPU, RAM or disk usage exceeds 80%. It also provides an explicitly labeled demonstration mode that generates simulated readings to illustrate changing measurements and alert behavior. The project demonstrates operating-system resource management, Linux virtual filesystems, file I/O and periodic monitoring.

1. Introduction

Operating systems manage CPU time, memory, processes and storage continuously. Linux exposes useful runtime measurements through a virtual /proc filesystem and standard system interfaces. Kernel Monitor collects several measurements in one terminal dashboard, helping learners observe system behavior without switching between multiple commands. It is a user-space monitoring program, not a kernel module.

2. Problem Statement

Checking resource usage manually often requires separate commands and interpretation of raw Linux system data. A simple consolidated display with periodic refresh and clear warnings can make basic system health easier to understand.

3. Objectives

• Display CPU, RAM, disk, process count and one-minute load average.

• Refresh readings automatically every second.

• Warn when CPU, RAM or disk usage exceeds 80%.

• Demonstrate Linux /proc, file operations, process enumeration and POSIX filesystem APIs.

• Provide a clearly identified simulated demonstration mode for explaining alerts.

4. Technology and Requirements

Component	Details
Operating system	Ubuntu or other compatible Linux
Language	C (C11)
Compiler	GCC

Component	Details
Interfaces	/proc/stat, /proc/meminfo, /proc/loadavg, /proc/PID, statvfs
Interface	Terminal / command line

5. System Architecture and Workflow

Ubuntu kernel statistics and filesystem APIs → C data-reading functions → usage calculations and process enumeration → threshold comparison → refreshed terminal dashboard. The main loop waits one second between refreshes. Live mode reads real operating-system data; demo mode substitutes randomly generated sample data and is labeled SIMULATED.

6. Module Descriptions

Module	Method and output
CPU	Read cumulative CPU counters from /proc/stat twice; calculate percentage from differences between successive samples.
RAM	Read MemTotal and MemAvailable from /proc/meminfo; calculate (total − available) / total × 100.
Disk	Use statvfs("/") to estimate root filesystem space unavailable to an unprivileged user.
Processes	Count numeric PID directories visible under /proc; this is a process count, not just currently CPU-running tasks.
Load average	Read first value of /proc/loadavg (one-minute load average, not a percentage).
Alerts	Print warning when CPU, RAM or disk reading is above 80%.

7. Build and Execution

Open a terminal in the project directory. Compile using:

gcc -Wall -Wextra -std=c11 KernelMonitor.c -o KernelMonitor

Run live monitoring: ./KernelMonitor

Run simulated demo: ./KernelMonitor --demo

Stop either mode using Ctrl+C.

8. Illustrative Output

LIVE (REAL UBUNTU DATA) | Refresh: 1 second | Update: 22

Measurement	Example
CPU usage	0.00%
RAM usage	7.31%
Disk usage	5.33%
Total processes	35
1-minute load	0.00
Alert	[NORMAL] No threshold exceeded

These values are an example from a previous Ubuntu run; they are not a performance benchmark. Actual readings vary with the system.

9. Testing and Observations

The program can be checked by compiling with GCC, running live mode and comparing values with system utilities such as top or free. Process counts can be observed when new programs start or finish. In demo mode, values vary by design and can cross the 80% warning threshold. In live mode, unchanged values are valid: disk space, process count and load average need not change every second.

10. Limitations

The monitor reads locally visible Linux resources only. CPU is calculated over a sampling interval; load average is smoothed. Process count includes all visible PIDs rather than only processes actively executing on a CPU. It has no persistent history, remote monitoring or desktop notification service. Demo values are simulated and must not be presented as actual measurements.

11. Future Scope

Possible enhancements include configurable alert thresholds, CSV logging, historical graphs, process detail views, and optional desktop notifications.

12. Conclusion

Kernel Monitor combines basic Linux resource readings into a single continuously refreshed terminal display. Its C implementation demonstrates operating-system interfaces and threshold-based monitoring, while the separate demo mode helps explain alert behavior during a classroom presentation.

References

Linux procfs documentation; Linux statvfs(3) manual; GCC documentation; project C source file KernelMonitor.c.

Team Members

Name	Roll Number
M. Subhash	2520030391
D. Namith	2520030249
E. Yeshwanth	2520030539